

EDA Final: Ejercicios y Soluciones Propias

Basado en MIT 6.851 Spring 2021 y slides locales

Como usar esta hoja

Cada ejercicio esta pensado para responderse a papel en una pagina: idea, pseudocodigo, prueba corta y complejidad. Los problem sets oficiales de MIT estan archivados localmente en `../originales/`; aqui solo se incluyen referencias y soluciones propias/resumidas.

1 Integer Data Structures

I1. vEB: tiempo de predecessor

Sea un universo de tamaño $u = 2^w$. Una estructura van Emde Boas divide cada clave x como $x = \langle c, i \rangle$, donde c es el cluster e i la posición dentro del cluster. Explica predecessor/successor y prueba el tiempo.

Solucion. Se guarda min, max, un *summary* de clusters no vacíos y una estructura por cluster. Para $\text{succ}(x = \langle c, i \rangle)$: si $x < \text{min}$, se responde min. Si el cluster c existe y $i < \max(\text{cluster}[c])$, se busca recursivamente dentro de ese cluster. Si no, se busca en **summary** el siguiente cluster no vacío c' y se responde $\langle c', \min(\text{cluster}[c']) \rangle$. Cada llamada reduce el universo a $\sqrt{u}$, entonces

$$T(u) = T(\sqrt{u}) + O(1) = O(\log \log u) = O(\log w).$$

La misma idea funciona simetricamente para predecessor.

I2. PS7 MIT: espacio de vEB con saving space

Referencia: `../originales/psets/ps7.pdf`. Estudia la version que guarda solo clusters no vacíos. Da una cota de espacio y como llegar a espacio lineal.

Solucion propia. La version sparse elimina clusters vacíos mediante hashing, pero una clave puede aparecer implicitamente en varios niveles por la ruta recursiva. Una cota natural es $O(n \log \log u)$ palabras si cada elemento causa presencia en un nivel por escala. Para espacio lineal, se usa indireccion: partir los n elementos en buckets de tamaño $\Theta(\log u)$ o $\Theta(w)$ mantenidos por estructuras simples, y construir vEB/x-fast/y-fast solo sobre representantes. El numero de representantes baja a $O(n/w)$ y cada bucket ocupa espacio proporcional a su tamaño; la suma queda $O(n)$ palabras, manteniendo busquedas con el costo del representante mas una busqueda local.

I3. x-fast versus y-fast

Compara x-fast y y-fast para predecessor en word RAM.

Solucion. x-fast guarda todos los prefijos de cada clave en tablas hash por nivel. Para buscar una clave x , hace binary search sobre la longitud del prefijo mas largo presente y luego usa punteros de hojas para obtener predecessor/successor. Tiempo $O(\log w)$, espacio $\Theta(nw)$.

y-fast aplica indireccion: solo los representantes de buckets estan en un x-fast. Cada bucket tiene tamaño esperado/amortizado $\Theta(w)$ y se guarda en una estructura balanceada local. Se consulta el representante en $O(\log w)$ y luego se busca dentro del bucket en $O(\log w)$. El espacio total baja a $O(n)$.

I4. PS8 MIT: compresion de signatures

Referencia: `../originales/psets/ps8.pdf`. Se tienen $k = \log^\epsilon n$ hashes de $\log n$ bits separados por ceros dentro de una palabra. Se quiere empaquetarlos juntos a la derecha en tiempo $O(1)$ word RAM.

Solucion propia. La herramienta conceptual es una multiplicacion por una mascara constante precomputada que mueve cada campo a su posicion objetivo, seguida de mascaras para eliminar basura. Como los campos no se solapan despues del desplazamiento, la suma/multiplicacion paralela dentro de palabra actua como una operacion SIMD: cada hash h_i se desplaza por una cantidad fija hasta quedar contiguo al siguiente. La prueba consiste en fijar una mascara con 1s en las posiciones de inicio de cada campo, multiplicar por un patron que copia esos campos a offsets controlados, y aplicar una mascara final de $k \log n$ bits. Todas las operaciones son de palabra, por lo que el tiempo es $O(1)$.

2 Static Trees, RMQ y LCA

S1. RMQ a LCA con Cartesian tree

Dado un arreglo $A[1..n]$, construye su Cartesian tree min-heap. Prueba que $\text{RMQ}_A(i, j)$ es el LCA de los nodos i y j .

Solucion. El Cartesian tree mantiene inorder igual al orden de indices y heap property por valor. Para cualquier intervalo $[i, j]$, el nodo de minimo valor en ese intervalo es el primer nodo del intervalo que separa el inorder en izquierda/derecha; por tanto es ancestro de todos los indices del intervalo. En particular, es el ancestro comun mas bajo de i y j . Si hubiera un ancestro comun mas bajo con menor profundidad dentro del intervalo, violaria la propiedad de heap o la separacion inorder. Entonces $\text{RMQ}_A(i, j) = \text{LCA}_T(i, j)$.

S2. LCA a ± 1 RMQ

Explica la reduccion de LCA en un arbol estatico a RMQ sobre un arreglo con diferencias consecutivas ± 1 .

Solucion. Hacer Euler tour del arbol y guardar dos arreglos: E , nodos visitados, y D , profundidades. Tambien guardar $\text{first}[v]$, primera aparicion de v en E . Para una query $\text{LCA}(u, v)$, sea $l = \min(\text{first}[u], \text{first}[v])$ y $r = \max(\dots)$. El LCA es el nodo $E[t]$ donde $D[t]$ es minimo en $[l, r]$. Como el Euler tour se mueve por una arista en cada paso, las profundidades cambian exactamente en $+1$ o -1 . Asi el problema se reduce a ± 1 RMQ.

S3. PS10 MIT: succinct ± 1 RMQ

Referencia: `../originales/psets/ps10.pdf`. Disena una estructura succinct para ± 1 RMQ con $n + o(n)$ bits y query constante.

Solucion propia. Usar dos niveles. Primero codificar el arreglo ± 1 por sus diferencias: un bit por paso, total $n + o(n)$ bits. Dividir en bloques pequenos de tamano $b = \frac{1}{2} \log n$. Cada bloque tiene solo $2^{b-1} = \sqrt{n}$ formas posibles; se precomputan tablas de respuestas internas para cada forma usando $o(n)$ bits. Luego agrupar bloques en super-bloques y guardar minimos de bloque en una macroestructura RMQ sparse/succinct sobre los minimos. Una query se parte en: sufijo del bloque izquierdo, bloques completos intermedios, prefijo del bloque derecho. Los extremos se responden por microtabla; el centro por macro RMQ. El espacio extra es $o(n)$ y el tiempo es $O(1)$.

S4. Imperial Roads

Se tiene un grafo ponderado. Para cada carretera consultada, se pide el peso del MST si esa carretera debe estar incluida.

Solucion. Construir un MST T con Kruskal y peso total W . Preprocesar T para responder $\text{maxEdge}(u, v)$ en el camino entre dos nodos, usando binary lifting, HLD o LCA con RMQ. Para una arista $e = (u, v, w)$, al agregarla a T se forma un ciclo. Para mantener un arbol con e , se debe eliminar una arista del camino $u \rightsquigarrow v$. La mejor eleccion es eliminar la de mayor peso. Por tanto

$$\text{ans}(e) = W + w - \text{maxEdge}(u, v).$$

Correctitud: por la propiedad del ciclo del MST, en cualquier ciclo, una arista de peso maximo puede excluirse de algun MST. Complejidad: Kruskal $O(m \log m)$, preproceso $O(n \log n)$, query $O(\log n)$ con binary lifting o $O(1)$ si se reduce a RMQ apropiadamente.

3 Strings

T1. Buscar un patron con suffix array

Dado texto fijo S y patron P , explica como reportar ocurrencias usando suffix array.

Solucion. Construir SA de $S\$$. Los sufijos que empiezan con P forman un intervalo contiguo en orden lexicografico, porque todos tienen prefijo comun P y ningun sufijo fuera del rango puede quedar entre ellos sin compartir el mismo prefijo. Hacer dos binary searches: primera posicion con sufijo $\geq P$ y primera posicion con sufijo $> P$ en orden prefijo. El intervalo resultante contiene las posiciones de ocurrencia. Tiempo de construccion segun algoritmo: $O(n \log n)$ con Manber-Myers o $O(n)$ con DC3/SA-IS; query directa $O(|P| \log n + \text{occ})$.

T2. Contar subcadenas distintas

Prueba la formula usando suffix array y LCP.

Solucion. El sufijo que empieza en i aporta $n - i$ prefijos, todos subcadenas de S . Si procesamos sufijos en orden de SA, el sufijo actual comparte exactamente LCP subcadenas-prefijo con el sufijo anterior; esas ya fueron contadas. Por tanto el numero de nuevas subcadenas aportadas es $(n - \text{SA}[i]) - \text{LCP}[i]$. Sumando:

$$\#distinct = \frac{n(n+1)}{2} - \sum_i \text{LCP}[i].$$

T3. Longest common substring

Dados strings A y B , encuentra la subcadena comun mas larga.

Solucion. Construir $S = A\#B\$$ con separadores que no aparecen en los strings. Construir SA y LCP. La respuesta es el maximo $\text{LCP}[i]$ tal que los sufijos $\text{SA}[i-1]$ y $\text{SA}[i]$ vienen de strings distintos. Si dos sufijos tienen un prefijo comun de longitud L , ese prefijo es una subcadena comun; el maximo entre pares adyacentes basta porque, en un conjunto de sufijos con prefijo comun, aparecen consecutivos en SA.

T4. Manber-Myers

Explica el algoritmo de doubling para construir suffix array.

Solucion. Mantener clases $\text{rank}_k[i]$ para subcadenas de longitud 2^k que empiezan en i . Inicialmente se ordena por caracter. Para pasar de k a $k+1$, ordenar cada posicion i por el par

$$(\text{rank}_k[i], \text{rank}_k[i + 2^k]).$$

Despues de ordenar, se reasignan clases. Cada iteracion duplica la longitud comparada; tras $\lceil \log n \rceil$ iteraciones, las clases identifican sufijos completos. Con counting/radix sort sobre ranks, cada iteracion cuesta $O(n)$ y el total es $O(n \log n)$.

T5. DC3 en una pagina

Da el esquema de DC3 y su recurrencia.

Solucion. Particionar posiciones por residuo modulo 3: S_0, S_1, S_2 . Ordenar recursivamente sufijos en $S_1 \cup S_2$ representando cada posicion por ternas de caracteres y convirtiendolas a ranks. Con esos ranks, ordenar S_0 por $(S[i], rank[i + 1])$. Luego hacer merge lineal entre S_0 y $S_1 \cup S_2$: para comparar $i \in S_0$ con $j \in S_1$, usar $(S[i], rank[i + 1])$; para $j \in S_2$, usar $(S[i], S[i + 1], rank[i + 2])$. La recurrencia es $T(n) = T(2n/3) + O(n) = O(n)$.

4 Dynamic Graph

D1. HLD path query

Prueba que HLD produce solo $O(\log n)$ cadenas en cualquier camino raiz-nodo y da path query.

Solucion. Una arista ligera hacia un hijo v cumple $size(v) \leq size(u)/2$. Cada vez que se baja por una arista ligera, el tamano del subarbol se reduce al menos a la mitad, asi que no puede ocurrir mas de $\log n$ veces en una ruta. Las aristas pesadas forman cadenas. Para query path u, v , mientras las cabezas de cadena sean distintas, subir la cabeza mas profunda y consultar el segmento $[in[head], in[node]]$. Al final ambos estan en la misma cadena y se consulta el segmento restante. Con segment tree, cada segmento cuesta $O(\log n)$ y hay $O(\log n)$ segmentos: total $O(\log^2 n)$.

D2. PS11 MIT: LCA con link-cut tree

Referencia: [../originales/psets/ps11.pdf](#). Explica como responder LCA en $O(\log n)$ usando link-cut trees.

Solucion propia. En un link-cut tree, `access(v)` expone la ruta desde la raiz representada hasta v como preferred path. Si primero hacemos `access(v)`, la ruta raiz- v queda marcada como la ultima ruta accedida. Luego hacemos `access(w)`. Durante este segundo acceso, el punto donde la busqueda cambia de una preferred path ya existente a la ruta hacia w identifica el ultimo vertice comun entre raiz- v y raiz- w , es decir, $LCA(v, w)$. Muchas implementaciones devuelven precisamente el ultimo nodo tocado/reconectado por `access(w)` despues de `access(v)`. Como cada access cuesta $O(\log n)$ amortizado, la query LCA tambien.

D3. Euler-tour tree para component sum

Mantener un forest dinamico con pesos en vertices: `link`, `cut`, `addVertex(v,x)` y `componentSum(v)`.

Solucion. Representar cada arbol por su Euler tour en un treap/BST implicito. Cada ocurrencia de vertice apunta al nodo del treap; guardar en cada treap la suma agregada de pesos. Para no contar un vertice multiples veces, se puede asignar el peso real solo a una ocurrencia canonica y peso cero a las demas. `link` y `cut` son secuencias de splits/merges en $O(\log n)$. Para `componentSum(v)`, tomar cualquier ocurrencia de v , encontrar la raiz del treap y leer su suma agregada. `addVertex` actualiza la ocurrencia canonica y recalcula agregados en $O(\log n)$.

D4. Fully dynamic connectivity general

Bosqueja la estructura con niveles y Euler-tour trees.

Solucion. Mantener niveles $0, \dots, \lfloor \log n \rfloor$. En cada nivel hay un spanning forest de las aristas de ese nivel o superiores, representado con Euler-tour trees. Las aristas tree mantienen conectividad; las non-tree son candidatas de reemplazo. Al borrar una arista que no es tree, solo se elimina. Al borrar una tree edge, el arbol se parte en dos componentes; se buscan aristas non-tree que reconecten ambas partes, promoviendo/demoviendo aristas por niveles para amortizar el costo. La intuicion del bound es que una arista solo puede bajar/subir $O(\log n)$ niveles y cada operacion de ETT cuesta $O(\log n)$, dando $O(\log^2 n)$ amortizado para updates.

Tabla final de complejidades

Tema	Estructura	Complejidad clave
Integer	vEB	$O(\log w)$ query/update, espacio grande
Integer	y-fast	$O(\log w)$ amortizado, espacio $O(n)$
Integer	fusion tree	$O(\log_w n)$ predecessor
Static trees	LCA via RMQ	$O(n)$ preprocess, $O(1)$ query
Strings	Manber-Myers	$O(n \log n)$ construccion SA
Strings	DC3/SA-IS	$O(n)$ construccion SA
Dynamic graph	HLD	$O(\log^2 n)$ path query/update
Dynamic graph	link-cut	$O(\log n)$ amortizado
Dynamic graph	Euler-tour tree	$O(\log n)$ link/cut/connectivity forest